



Iby and Aladar Fleischman  
Faculty of Engineering  
Tel Aviv University

הפקולטה להנדסה  
ע"ש איבי ואלדר פליישר  
אוניברסיטת תל-אביב

# Target Range Estimation Using Self-Position and Camera Angle

Project Number: 25-1-1-3269

## Project Report

Student: Shoval Safir ID: 322735945

Student: Daniel Nethanel ID: 214194847

Supervisor: Dr. Gavriel Davidov

Project Carried Out at: Tel Aviv University

## Contents

|                                                               |    |
|---------------------------------------------------------------|----|
| Abstract.....                                                 | 4  |
| 1. Introduction .....                                         | 5  |
| 1.1 Motivation .....                                          | 5  |
| 1.2 Project Goals .....                                       | 5  |
| 1.3 Approach to Solving the Problem .....                     | 5  |
| 1.4 Evaluated Algorithms .....                                | 6  |
| 2. Theoretical Background.....                                | 6  |
| 2.1 The General Geometric Scenario .....                      | 6  |
| 2.2 Recursive Least Squares (RLS).....                        | 8  |
| 2.3 The Kalman Filter .....                                   | 9  |
| 2.4 Extended Kalman Filter (EKF) .....                        | 11 |
| 2.5 Unscented Kalman Filter (UKF) .....                       | 12 |
| 2.6 Coordinate Conversion for Real-World Implementation ..... | 14 |
| 3. System Design and Simulation.....                          | 15 |
| 3.1 Overview .....                                            | 15 |
| 3.2 Kinematic Modeling (Observer and Target).....             | 15 |
| 3.3 Bearing Calculation and Signal Unwrapping.....            | 17 |
| 3.4 Algorithm Implementation and Routing .....                | 18 |
| 3.5 Performance Evaluation and Error Metrics .....            | 19 |
| 3.6 Complete System Architecture .....                        | 21 |
| 3.7 Simulation Results.....                                   | 21 |
| 3.8 Chapter Summary .....                                     | 26 |
| 4. Implementation .....                                       | 26 |
| 4.1 Field Data Acquisition.....                               | 26 |
| 4.2 Hardware Limitations: Gimbal Magnetic Interference .....  | 27 |
| 4.3 Engineering Workaround: Gyroscope Integration .....       | 28 |
| 4.4 Data Preprocessing and Coordinate Transformation.....     | 28 |
| 4.5 Simulink Integration.....                                 | 29 |
| 5. Experimental Results and Performance Analysis.....         | 29 |
| 5.1 Sensor Configuration and Adaptation .....                 | 29 |
| 5.2 Performance Comparison in Representative Scenarios .....  | 30 |
| 5.3 Discussion and Proof of Concept.....                      | 33 |
| 6. Summary, Conclusions, and Suggestions for Future Work..... | 34 |
| 6.1 Summary .....                                             | 34 |
| 6.2 Conclusions .....                                         | 35 |
| 6.3 Suggestions for Future Work .....                         | 35 |
| 7. Documentation .....                                        | 36 |

|                      |    |
|----------------------|----|
| 8. Bibliography..... | 37 |
|----------------------|----|

## Tables

|                                                                                                                    |    |
|--------------------------------------------------------------------------------------------------------------------|----|
| Table 1: Comparison of filter convergence and CPU execution times between simulation and real-world scenarios..... | 33 |
|--------------------------------------------------------------------------------------------------------------------|----|

## Figures

|                                                                                                                       |    |
|-----------------------------------------------------------------------------------------------------------------------|----|
| Figure 1: Overall system block diagram .....                                                                          | 4  |
| Figure 2: Simulink block diagram of the Coordinate Generation subsystem .....                                         | 16 |
| Figure 3 : Simulink block diagram for relative bearing calculation and signal unwrapping .....                        | 17 |
| Figure 4: Simulink EKF Algorithm architecture.....                                                                    | 18 |
| Figure 5: Simulink block diagram of the error calculation subsystem .....                                             | 20 |
| Figure 6: Position tracking comparison for RLS, EKF, and UKF in Scenario A (Static Target) .....                      | 22 |
| Figure 7: Distance estimation error (%) vs. Time comparison for RLS, EKF, and UKF in Scenario A (Static Target).....  | 22 |
| Figure 8: Position tracking comparison for RLS, EKF, and UKF in Scenario B (Dynamic Target).....                      | 24 |
| Figure 9: Distance estimation error (%) vs. Time comparison for RLS, EKF, and UKF in Scenario B (Dynamic Target)..... | 24 |
| Figure 10: Field testing environment (roller skating rink) and observer platform (bicycle).....                       | 27 |
| Figure 11: Smartphone observer mounted on the tracking gimbal .....                                                   | 27 |
| Figure 12: Smoothed observer path and true target coordinates in ENU system (Static Scenario).....                    | 30 |
| Figure 13: Measured vs. theoretical bearing angle (degrees) over time (Static Scenario).....                          | 31 |
| Figure 14: Distance estimation error (%) vs. Time comparison for RLS and UKF/EKF (Static Scenario).....               | 31 |
| Figure 15: Figure-eight observer path and linear target path in local ENU system (Dynamic Scenario).....              | 32 |
| Figure 16: Measured vs. theoretical bearing angle (degrees) over time (Dynamic Scenario) .....                        | 32 |
| Figure 17: Distance estimation error (%) vs. Time comparison for RLS and UKF/EKF (Dynamic Scenario) .....             | 33 |

## Abstract

This project evaluates a passive target range estimation system using self-position and bearing-angle measurements. Because passive tracking operates silently, it is highly advantageous over active radar for naval and military security. The study assesses Recursive Least Squares (RLS), Extended Kalman Filter (EKF), and Unscented Kalman Filter (UKF) algorithms through theoretical MATLAB/Simulink simulations and live field tests. During real-world trials, gimbal-induced magnetic interference required an engineering workaround: bypassing the magnetometer and integrating gyroscope angular velocity over time to compute relative bearing angles.

Results show all algorithms succeed under ideal conditions. Against physical noise, EKF and UKF demonstrate superior robustness, converging with under 10% error within 2.06 seconds for dynamic targets. RLS converges quickly for static targets (8.02s) but diverges for moving ones. With an execution time of  $<10\ \mu\text{s}$ , EKF proved optimal for real-time deployment. As illustrated in Figure 1, observer coordinates  $(x, y)$  and relative bearing  $\beta$  are preprocessed, aligned, and converted to the local ENU coordinate system prior to filter estimation.

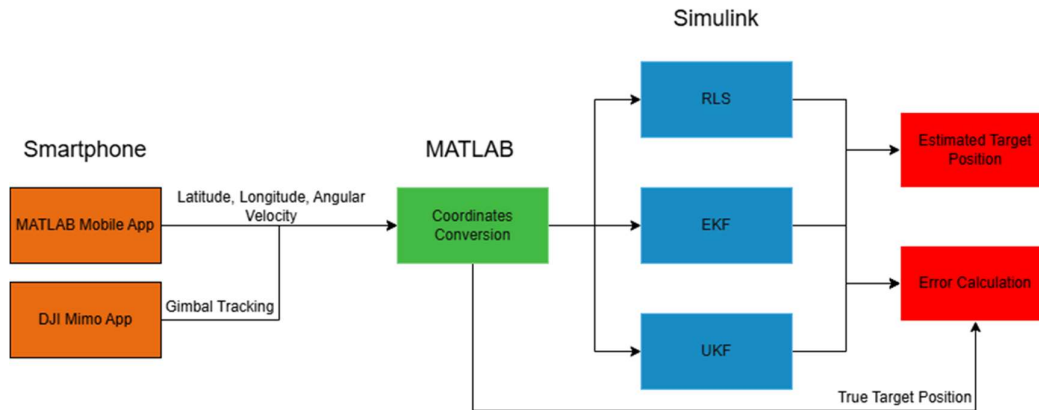


Figure 1: Overall system block diagram

# 1. Introduction

## 1.1 Motivation

The central motivation for this project stems from real-world military and naval security scenarios. Active detection methods, such as emitting radar or sonar signals, fundamentally compromise a vessel's position and expose it to enemy targeting. Passive tracking provides a critical alternative, allowing a ship to locate a target without broadcasting its own location. This approach of gathering intelligence solely through passive emissions is highly applicable across multiple domains, including submarine acoustic tracking, autonomous navigation, and bearing-only Simultaneous Localization and Mapping (SLAM).

## 1.2 Project Goals

The primary objective of this project is to implement and evaluate a passive target range estimation system utilizing self-position and bearing-angle measurements. The project quantitatively analyzes the performance of three different passive estimation algorithms under various conditions.

To establish a strict baseline for evaluating the algorithms during both the simulation and live-testing phases, the system is designed to meet three specific quantitative performance metrics:

- **Relative Position Error:** A maximum position error of up to  $\sim 10\%$  relative to the true target range, which evaluates the final tracking quality after the filter has converged.
- **Algorithm Convergence Time:** Because frequent angular sampling without sufficient observer movement provides redundant data (rendering the range unobservable), the convergence time naturally scales with the target's distance. The observer requires time to traverse a large enough distance to generate meaningful geometric angular changes. The convergence thresholds are strictly set to  $\sim 20$  seconds for ranges of tens of meters,  $\sim 30$  seconds for hundreds of meters, and  $\sim 45$  seconds for ranges of several kilometers.
- **Computational Efficiency:** A maximum execution time of 10 microseconds per single filter iteration. This guarantees the algorithms remain efficient enough to process continuous data streams in real-time without inducing system lag or processing bottlenecks.

## 1.3 Approach to Solving the Problem

The project methodology is divided into two primary phases, moving from theoretical simulation to practical real-world testing:

- **Phase 1 (Simulation):** The first semester focuses on comprehensive theoretical research and software implementation within the MATLAB and Simulink environments. The system inputs simulated target and observer data to test the estimation algorithms and record their theoretical performance.
- **Phase 2 (Implementation and Live Testing):** The second semester transitions into physical data collection using a smartphone mounted on a DJI Osmo Mobile 6 gimbal. The MATLAB Mobile App is utilized to acquire live GPS coordinates and compass bearing data. This raw geographic data is converted into a local metric coordinate system and fed directly into the Simulink models to compare real-world performance against the Phase 1 simulations.

## 1.4 Evaluated Algorithms

To determine the most effective method for bearing-only estimation, the system evaluates the following three algorithms:

- **Recursive Least Squares (RLS):** An adaptive technique designed for linear systems that recursively updates the parameter estimation with each new measurement step, aiming to minimize the Mean Squared Error (MSE).
- **Extended Kalman Filter (EKF):** An extension of the classic Kalman Filter adapted for non-linear systems. It utilizes a first-order Taylor series expansion (Jacobian matrices) to linearize the system dynamics around the current estimate.
- **Unscented Kalman Filter (UKF):** An advanced non-linear filter that avoids the complex Jacobian calculations of the EKF. Instead, it uses the Unscented Transform, selecting specific "Sigma Points" to accurately capture and propagate the mean and covariance of the probability distribution, offering higher numerical stability for highly non-linear problems.

## 2. Theoretical Background

### 2.1 The General Geometric Scenario

To establish the tracking models, we must first define the local Cartesian coordinate system representing the operational theater. The fundamental challenge of passive tracking is estimating the range to a target using only the measured bearing angles and the known kinematics of the observer.

Let us define the states of the two primary entities in a two-dimensional space (East-North):

- **Observer (Vessel):** Denoted with the subscript  $i$ , characterized by its position  $(x_i, y_i)$  and velocity  $v_i$ .
- **Target:** Denoted with the subscript  $t$ , characterized by its position  $(x_t, y_t)$  and velocity  $v_t$ .

The core measurement in this passive system is the relative bearing angle,  $\beta$ . This is defined as the angle measured from a reference axis (typically the x-axis, representing East) to the line-of-sight connecting the observer to the target.

At any discrete time step  $k$ , the geometric relationship between the observer, the target, and the measured bearing angle is given by:

$$(1) \quad \tan(\beta) = \frac{y_t - y_i}{x_t - x_i}$$

This function is highly non-linear, which immediately complicates the estimation process. As established by Aidala [1], attempting to solve this non-linear system while the observer moves in a constant-velocity, straight-line trajectory results in an unobservable system. In such a scenario, the filter cannot mathematically decouple the target's range from its velocity.

To achieve system observability and successfully converge on the target's true position, the observer must execute a maneuver—specifically a non-linear trajectory, such as an arc or circular pattern—to introduce sufficient geometric diversity into the measurements.

### 2.1.1 Measurement Linearization

While advanced filters like the EKF and UKF handle non-linearities directly, it is beneficial to manipulate the basic geometric equation into a pseudo-linear form. This manipulation forms the mathematical basis for linear estimators like the Recursive Least Squares (RLS) algorithm.

We begin by expanding the tangent function:

$$(2) \quad \frac{\sin(\beta)}{\cos(\beta)} = \frac{y_t - y_i}{x_t - x_i}$$

Cross-multiplying the terms yields:

$$(3) \quad \sin(\beta) \cdot (x_t - x_i) = \cos(\beta) \cdot (y_t - y_i)$$

Rearranging the variables to isolate the known observer states on one side and the unknown target states on the other results in a linear relationship:

$$(4) \quad y_i \cos(\beta) - x_i \sin(\beta) = y_t \cos(\beta) - x_t \sin(\beta)$$

This final equation allows us to represent the dynamic tracking problem under the framework of linear systems, provided the target remains static or we construct a specific state vector for its kinematics. We can now present the mathematical models for each of the three evaluated estimation algorithms utilizing this geometric foundation.

## 2.2 Recursive Least Squares (RLS)

The Recursive Least Squares (RLS) algorithm is an adaptive filter used to estimate the parameters of a linear system. It updates its parameter estimates recursively as each new data point becomes available, aiming to minimize the expected value of the squared error. This makes it an optimal estimator in the Mean Squared Error (MSE) sense [10]. Because RLS requires a linear model, we apply it directly to the pseudo-linear measurement model developed in the previous section.

### 2.2.1 RLS for a Static Target

For a stationary target, the state vector we wish to estimate at time step  $k$  contains the target's 2D coordinates:

$$(5) \quad \theta_k = \begin{pmatrix} x_{t,k} \\ y_{t,k} \end{pmatrix}$$

Using the linearized geometric relationship from Equation (4), we define our measurement vector  $z_k$  based solely on the known observer kinematics and the measured bearing:

$$(6) \quad z_k = y_{i,k} \cos(\beta_k) - x_{i,k} \sin(\beta_k)$$

We also define the observation vector,  $\Phi_k$ , which maps the state to the measurement:

$$(7) \quad \Phi_k = \begin{pmatrix} -\sin(\beta_k) \\ \cos(\beta_k) \end{pmatrix}$$

This allows us to represent the measurement equation in a standard linear form:

$$(8) \quad z_k = \Phi_k^T \theta_k$$

The RLS algorithm iteratively refines the estimate  $\hat{\theta}_k$  by calculating the estimation error  $e_k$ , defined as the difference between the actual measurement and the predicted measurement:

$$(9) \quad e_k = z_k - \Phi_k^T \hat{\theta}_k$$

To correct the estimate, the algorithm computes a gain vector,  $g_k$ . This vector determines the weight of the new measurement.  $P_k$  represents the state



covariance matrix, which acts as a measure of confidence in the current estimate (with smaller values indicating higher certainty):

$$(10) \quad g_k = P_k \Phi_k (I + \Phi_k^T P_k \Phi_k)^{-1}$$

The covariance matrix is then updated to reflect the newly acquired information, decreasing as the algorithm converges:

$$(11) \quad P_{k+1} = P_k - g_k \Phi_k^T P_k$$

Finally, the state estimate for the next time step is updated using the computed gain and error:

$$(12) \quad \hat{\theta}_{k+1} = \hat{\theta}_k + g_k e_k$$

### 2.2.2 RLS for a Dynamic Target (Forgetting Factor)

When tracking a dynamic target, the static RLS model must be modified. The standard static algorithm assigns equal weight to all historical measurements, which means that for a moving target, obsolete older measurements will distort the current position estimate [3].

To address this, we introduce a "forgetting factor",  $\lambda$ . A value of  $\lambda = 1$  reverts the algorithm to the static model (infinite memory), whereas lower values give exponentially less weight to older data, allowing the filter to "forget" past states and track dynamic changes. In our implementation, to smoothly handle noise, the forgetting factor is annealed over time. It dynamically updates per iteration utilizing a specific growth rate ( $\lambda_{rate} = 0.8$ ) and an initial value ( $\lambda_{initial} = 0.95$ ):

$$(13) \quad \lambda_k = \lambda_{rate} \cdot \lambda_{k-1} + (1 - \lambda_{rate}) = 1 + \lambda_{rate}^k \cdot (\lambda_{initial} - 1)$$

Incorporating the forgetting factor requires adjusting the gain vector and covariance update equations to scale the historical data:

$$(14) \quad g_k = P_k \Phi_k (\lambda_k I + \Phi_k^T P_k \Phi_k)^{-1}$$

$$(15) \quad P_{k+1} = \frac{1}{\lambda_k} (P_k - g_k \Phi_k^T P_k)$$

## 2.3 The Kalman Filter

The Kalman Filter is a mathematical algorithm designed to optimally estimate the hidden states of a linear dynamic system using a series of noisy measurements observed over time [11]. The algorithm operates on a fundamental two-model framework:

**The Process Model:** Describes how the system state transitions from one time step to the next.

$$(16) \quad x_k = F_k x_{k-1} + w_k$$

**The Measurement Model:** Describes the relationship between the system state and the actual measurements received.

$$(17) \quad z_k = H_k x_k + v_k$$

Where:

- $F_k$  is the state transition matrix.
- $H_k$  is the measurement (or observation) matrix.
- $x_k$  is the state vector we wish to estimate.
- $w_k \sim \mathcal{N}(0, Q_k)$  represents the process noise, assumed to be zero-mean Gaussian with covariance matrix  $Q_k$ .
- $z_k$  is the measurement vector at time  $k$ .
- $v_k \sim \mathcal{N}(0, R_k)$  represents the measurement noise, assumed to be zero-mean Gaussian with covariance matrix  $R_k$ .

The objective is to find the optimal linear estimate of  $x_k$  given all measurements up to time  $k$ . The algorithm achieves this through a continuous two-step predictor-corrector cycle:

### Step 1: Prediction (Time Update)

This step projects the current state and error covariance forward in time to obtain the *a priori* estimates for the next time step.

$$(18) \quad \hat{x}_{k|k-1} = F_k \hat{x}_{k-1|k-1}$$

$$(19) \quad P_{k|k-1} = F_k P_{k-1|k-1} F_k^T + Q_k$$

Where  $P_k$  is the error covariance matrix, representing the uncertainty in the estimate.

### Step 2: Update (Measurement Update)

Once a new measurement is observed, the algorithm calculates the Kalman Gain ( $K_k$ ), which dictates how much weight to give the new measurement versus the model's prediction. If the measurement is highly noisy (high  $R_k$ ), the gain is low, and the filter trusts its prediction more. If the prediction is highly uncertain (high  $P_k$ ), the gain is high, and the filter trusts the new measurement more.

$$(20) \quad K_k = P_{k|k-1} H_k^T \cdot (H_k P_{k|k-1} H_k^T + R_k)^{-1}$$

The state estimate is then corrected using the actual measurement  $z_k$  to produce the *a posteriori* estimate:

$$(21) \quad \hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k (z_k - H_k \hat{x}_{k|k-1})$$

Finally, the error covariance is updated to reflect the increased certainty after incorporating the measurement:

$$(22) \quad P_{k|k} = (I - K_k H_k) P_{k|k-1}$$

### The Non-Linearity Problem

The standard Kalman Filter requires that both the process and measurement models be strictly linear combinations of the state variables. However, as established in Equation (1), our bearing measurement relies on the arctangent function (or tangent ratios), which is highly non-linear.

Because our system cannot be accurately modeled with linear equations, the basic Kalman Filter will fail to converge. Therefore, we must implement advanced extensions of this algorithm – specifically, the Extended Kalman Filter (EKF) and the Unscented Kalman Filter (UKF) – which are mathematically designed to handle non-linear dynamics.

## 2.4 Extended Kalman Filter (EKF)

The Extended Kalman Filter (EKF) is a standard extension of the Kalman Filter specifically designed for non-linear systems. It operates by linearizing the non-linear process and measurement models around the current state estimate using a first-order Taylor series expansion, effectively computing the Jacobian matrices [5]. Like the linear Kalman Filter, the EKF operates in a predictor-corrector loop to minimize the Mean Squared Error (MSE).

### 2.4.1 EKF for a Static Target

Unlike the pseudo-linear RLS model, the EKF models the non-linear relationship directly. We define the state vector  $x_k$  to include the position of the static target, as well as the position and velocity of the moving observer:

$$(23) \quad x_k = (x_t \ y_t \ x_i \ \dot{x}_i \ y_i \ \dot{y}_i)^T$$

The measurement vector  $z_k$  incorporates the observer's known coordinates alongside the zeroed outcome of our rearranged geometric bearing equation:

$$(24) \quad z_k = (x_i \ y_i \ 0)^T$$

To handle the non-linearities, the EKF calculates the Jacobian of the process function ( $F_k$ ) and the measurement function ( $H_k$ ) at each time step. Because the observer moves at a constant velocity and the target is static, the process model is linear, yielding a constant state transition matrix:

$$(25) \quad F_k = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & \Delta t & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & \Delta t \\ 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

However, the measurement model is non-linear due to the bearing angle  $\beta$ . The corresponding Jacobian measurement matrix  $H_k$  evaluates to:

$$(26) \quad H_k = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ \sin(\beta_k) & -\cos(\beta_k) & -\sin(\beta_k) & 0 & \cos(\beta_k) & 0 \end{pmatrix}$$

By utilizing these Jacobians, the EKF can apply the standard Kalman Filter prediction and update equations detailed in Section 2.3 to successfully converge on the target's range.

#### 2.4.2 EKF for a Dynamic Target

When the target is in motion, the static assumption is broken. The state vector is expanded to include the target's velocity components:

$$(27) \quad x_k = (x_t \quad \dot{x}_t \quad y_t \quad \dot{y}_t \quad x_i \quad \dot{x}_i \quad y_i \quad \dot{y}_i)^T$$

The Jacobian matrices  $F_k$  and  $H_k$  are subsequently expanded to accommodate the dynamic target components, allowing the filter to estimate the target's trajectory in addition to its range.

## 2.5 Unscented Kalman Filter (UKF)

The Unscented Kalman Filter (UKF) is a more advanced non-linear estimator. Instead of using Taylor series approximations (Jacobians) which can introduce significant errors in highly non-linear models, the UKF utilizes a deterministic sampling technique known as the Unscented Transform [9]. It selects a minimal set of sample points (Sigma Points) that accurately captures the true mean and covariance of the state distribution, and propagates these points directly through the non-linear functions.

#### 2.5.1 UKF Core Algorithm

For a state vector of dimension  $L$  (where  $L = 6$  for a static target and  $L = 8$  for a dynamic target), the UKF executes the following four-step recursive loop:

##### Step 1: Sigma Point and Weight Generation

We define a scaling parameter  $\lambda = \alpha^2(L + \kappa) - L$ , where  $\alpha$  determines the spread of the points and  $\kappa$  is a secondary scaling factor. The associated weights for calculating the mean ( $\eta^m$ ) and covariance ( $\eta^c$ ) are defined as:

$$(28) \quad \eta_0^m = \frac{\lambda}{L + \lambda}, \quad \eta_0^c = \frac{\lambda}{L + \lambda} + 1 - \alpha^2 + \beta$$

$$(29) \quad \eta_i^m = \eta_i^c = \frac{1}{2(L + \lambda)} \quad \text{for } i = 1, \dots, 2L$$

Using the Cholesky decomposition of the error covariance matrix,  $2L + 1$  Sigma Points ( $\chi$ ) are generated:

$$(30) \quad \chi_{k-1} = [\hat{x}_{k-1} \quad \hat{x}_{k-1} + \sqrt{L + \lambda} \sqrt{P_{k-1}} \quad \hat{x}_{k-1} - \sqrt{L + \lambda} \sqrt{P_{k-1}}]$$

### Step 2: State Prediction

Each Sigma Point is propagated through the non-linear process model,  $f(x)$ :

$$(31) \quad \chi_{k|k-1}^{(i)} = f(\chi_{k-1}^{(i)}) \quad \text{for } i = 0, \dots, 2L$$

The predicted state mean and covariance are then calculated using the weighted sums:

$$(32) \quad \hat{x}_{k|k-1} = \sum_{i=0}^{2L} \eta_i^m \chi_{k|k-1}^{(i)}$$

$$(33) \quad P_{k|k-1} = Q_{k-1} + \sum_{i=0}^{2L} \eta_i^c (\chi_{k|k-1}^{(i)} - \hat{x}_{k|k-1}) (\chi_{k|k-1}^{(i)} - \hat{x}_{k|k-1})^T$$

### Step 3: Measurement Prediction

The predicted Sigma Points are passed through the non-linear measurement model,  $h(x)$ :

$$(34) \quad \Psi_{k|k-1}^{(i)} = h(\chi_{k|k-1}^{(i)}) \quad \text{for } i = 0, \dots, 2L$$

The expected measurement mean ( $\hat{y}_{k|k-1}$ ), the measurement covariance matrix ( $P_k^{yy}$ ), and the state-measurement cross-covariance matrix ( $P_k^{xy}$ ) are computed:

$$(35) \quad \hat{y}_{k|k-1} = \sum_{i=0}^{2L} \eta_i^m \Psi_{k|k-1}^{(i)}$$

$$(36) \quad P_k^{yy} = R_k + \sum_{i=0}^{2L} \eta_i^c (\Psi_{k|k-1}^{(i)} - \hat{y}_{k|k-1}) (\Psi_{k|k-1}^{(i)} - \hat{y}_{k|k-1})^T$$

$$(37) \quad P_k^{xy} = \sum_{i=0}^{2L} \eta_i^c (\chi_{k|k-1}^{(i)} - \hat{x}_{k|k-1}) (\Psi_{k|k-1}^{(i)} - \hat{y}_{k|k-1})^T$$

#### Step 4: State Update

Finally, the Kalman Gain ( $K_k$ ) is calculated, and the state and covariance matrices are updated using the actual received measurement,  $z_k$ :

$$(38) \quad K_k = P_k^{xy} (P_k^{yy})^{-1}$$

$$(39) \quad \hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k (z_k - \hat{y}_{k|k-1})$$

$$(40) \quad P_{k|k} = P_{k|k-1} - K_k P_k^{yy} K_k^T$$

## 2.6 Coordinate Conversion for Real-World Implementation

To transition the theoretical models from simulation to physical live testing (Phase 2), real-world data must be mapped to the Cartesian system utilized by the Simulink algorithms. The physical tracker collects data via the smartphone's GPS as Geodetic coordinates (Latitude  $\varphi$ , Longitude  $\lambda$ , Altitude  $h$ ) [6].

Because the estimation algorithms require a localized metric grid, this geodetic data must undergo a two-step mathematical transformation into a local East-North-Up (ENU) Cartesian system. While MATLAB's built-in functions facilitate this in the software, the underlying mathematics are defined below.

### 1. Geodetic to Earth-Centered, Earth-Fixed (ECEF)

The GPS data is first converted to the ECEF global Cartesian system ( $X$ ,  $Y$ ,  $Z$ ) using the *WGS84* reference ellipsoid [8]. Using the semi-major axis  $a$  and eccentricity  $e$ , the prime vertical radius of curvature  $N$  is calculated:

$$(41) \quad N = \frac{a}{\sqrt{1 - e^2 [\sin(\varphi)]^2}}$$

The ECEF coordinates are then extracted:

$$(42) \quad X = (N + h) \cos(\varphi) \cos(\lambda)$$

$$(43) \quad Y = (N + h) \cos(\varphi) \sin(\lambda)$$

$$(44) \quad Z = ((1 - e^2)N + h) \sin(\varphi)$$

### 2. ECEF to East-North-Up (ENU)

To create a localized grid measured in meters (where the X-axis points East and the Y-axis points North), the ECEF coordinates are shifted relative to a defined local reference point ( $X_0$ ,  $Y_0$ ,  $Z_0$ ) [7]:

$$(45) \quad \Delta X = X - X_0$$

$$(46) \quad \Delta Y = Y - Y_0$$

$$(47) \quad \Delta Z = Z - Z_0$$

Finally, this displacement vector is multiplied by a rotation matrix derived from the reference point's latitude ( $\varphi_0$ ) and longitude ( $\lambda_0$ ) to align with the local tangent plane:

$$(48) \quad \begin{pmatrix} x_{East} \\ y_{North} \\ z_{Up} \end{pmatrix} = \begin{pmatrix} -\sin(\lambda_0) & \cos(\lambda_0) & 0 \\ -\sin(\varphi_0)\cos(\lambda_0) & -\sin(\varphi_0)\sin(\lambda_0) & \cos(\varphi_0) \\ \cos(\varphi_0)\cos(\lambda_0) & \cos(\varphi_0)\sin(\lambda_0) & \sin(\varphi_0) \end{pmatrix} \begin{pmatrix} \Delta X \\ \Delta Y \\ \Delta Z \end{pmatrix}$$

This  $x_{East}$  and  $y_{North}$  data perfectly matches the geometric boundaries ( $x$ ,  $y$ ) defined in Section 2.1, allowing the real-world gimbal data to be fed directly into the RLS, EKF, and UKF Simulink blocks.

## 3. System Design and Simulation

### 3.1 Overview

The first phase of the project focuses on constructing a robust, purely theoretical simulation environment using MATLAB and Simulink. Before introducing the unpredictability of live hardware sensors, this software architecture allows us to systematically validate the correctness, efficiency, and convergence of the RLS, EKF, and UKF algorithms under strictly controlled kinematic scenarios.

### 3.2 Kinematic Modeling (Observer and Target)

To rigorously evaluate the system's observability and the filters' tracking capabilities, we engineered custom data-generation blocks that simulate the movement of both the observer vessel and the target.

While constant-velocity linear trajectories often lead to unobservable states (as established in Chapter 2), introducing maneuvering is mathematically necessary. To test this, our simulation supports switching between multiple kinematic profiles. Advancing beyond standard linear and circular paths, our model specifically incorporates a continuous lemniscate (figure-eight) trajectory. This provides a highly non-linear, complex maneuvering profile to stress-test the algorithms' convergence capabilities.

The generated coordinates ( $x$ ,  $y$ ) for either the observer or the target at any simulation time  $t$  are calculated based on the selected mode:

- **Static:** Fixed coordinate points ( $x_0, y_0$ ).

- **Linear:** Constant velocity movement where  $x(t) = x_0 + v_x t$  and  $y(t) = y_0 + v_y t$ .
- **Circular:** Constant angular velocity movement defined by a radius  $R$  and frequency  $\omega$ , where  $x(t) = x_0 + R \cos(\omega t)$  and  $y(t) = y_0 + R \sin(\omega t)$ .
- **Lemniscate:** A complex figure-eight pattern adding continuous variation in both heading and acceleration, where  $x(t) = \frac{a \cos(\omega t)}{1 + \sin^2(\omega t)}$  and  $y(t) = \frac{a \sin(\omega t) \cos(\omega t)}{1 + \sin^2(\omega t)}$ .

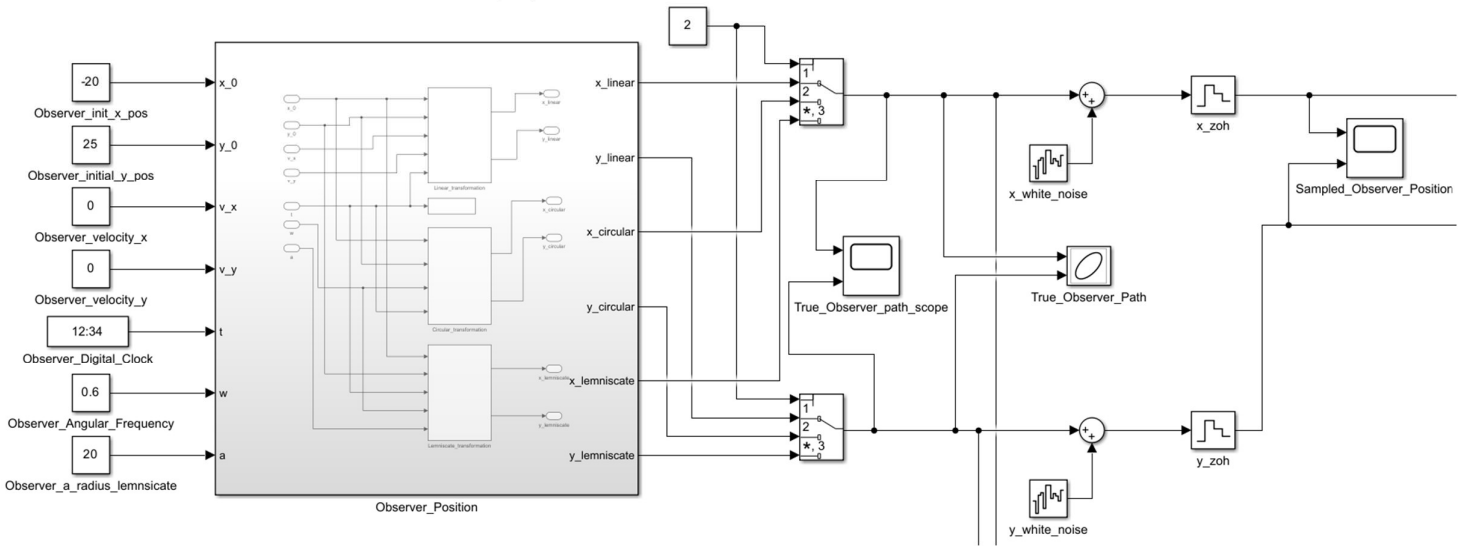


Figure 2: Simulink block diagram of the Coordinate Generation subsystem

As illustrated in the provided diagram, the continuous coordinates are generated in parallel within the central *Observer\_Position* subsystem.

The user defines the initial state constants (position, linear velocities, angular frequency, and the lemniscate scale factor  $a$ ). Multiport switches are then used to dynamically route the desired kinematic profile into the main simulation bus.

Crucially, to bridge the gap between an idealized mathematical environment and real-world implementation, we immediately corrupt this perfect trajectory data. Band-limited white noise is injected into the true X and Y coordinates to simulate the inherent inaccuracies of physical GPS sensors. Finally, Zero-Order Hold (ZOH) blocks discretize these continuous, noisy signals to replicate the specific polling rates of physical tracking hardware. This ensures the algorithms are tested against realistic, fragmented data streams.

(Note: The exact same architectural logic is duplicated in a parallel block to generate the target's position).



### 3.3 Bearing Calculation and Signal Unwrapping

With both the observer and target coordinates generated and sampled, the system must calculate the core measurement of the project: the relative bearing angle ( $\beta$ ).

Using the arctangent function, the Simulink model continuously computes the angle between the two vessels. However, using inverse trigonometric functions computationally introduces a critical discontinuity. When the calculated angle crosses the boundaries of  $\pm\pi$  radians, the output artificially "jumps," creating a sharp, non-physical spike in the data. If fed directly into the estimation algorithms, these discontinuities would cause the filters to diverge immediately.

To resolve this, the raw arctangent output must be passed through a dedicated Unwrapping block, which monitors the signal for sudden jumps exceeding a predefined threshold (e.g.,  $\pi$ ). When a jump is detected, the block adds or subtracts  $2\pi$ , ensuring the bearing angle remains continuous and accurately reflects the physical rotation of the observer relative to the target.

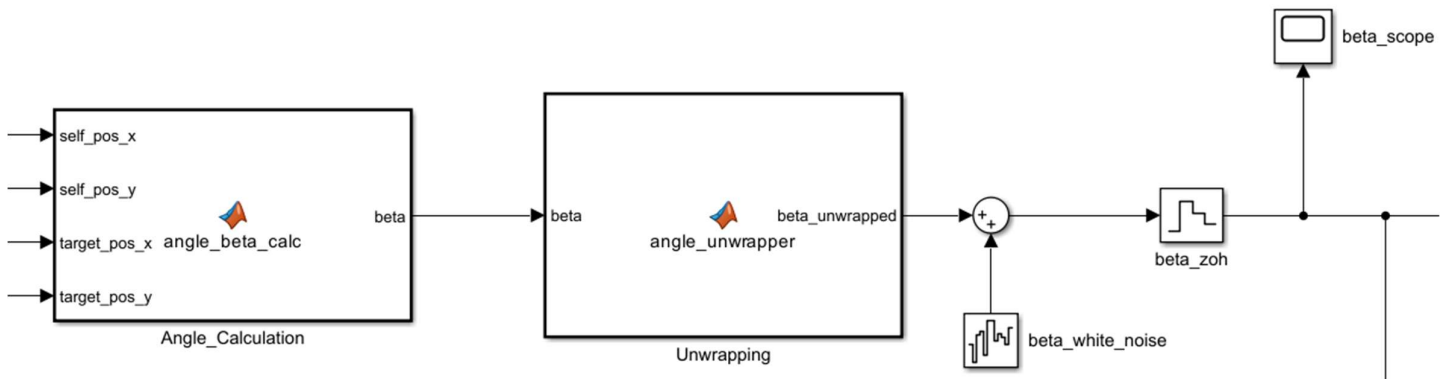


Figure 3 : Simulink block diagram for relative bearing calculation and signal unwrapping

As shown in the provided model, the raw positional data for both the observer and target are fed into a dedicated MATLAB function block (*Angle\_Calculation*) to compute the true relative bearing. The resulting angle,  $\beta$ , is immediately passed through a custom Unwrapping function block to resolve the trigonometric ambiguities and ensure signal continuity.

Mirroring the treatment of the observer's positional data, this continuous, unwrapped bearing signal is then corrupted with Gaussian white noise (*beta\_white\_noise*). This step is vital to realistically emulate the inherent inaccuracies and drift of a physical compass or optical bearing sensor. Finally, a Zero-Order Hold (*beta\_zoh*) block discretizes the signal. At this stage, the simulated measurement data is fully prepared and perfectly replicates the fragmented, noisy inputs the system will encounter during live field testing.

### 3.4 Algorithm Implementation and Routing

With the simulated inputs generated, noise-corrupted, and properly sampled, the data must be routed into the actual estimation filters. The system requires the observer's self-position  $(x_i, y_i)$  and the measured bearing angle  $\beta$  to execute the position estimation. To conduct a comprehensive comparison, these three variables are fed simultaneously into the parallel algorithm blocks: RLS, EKF, and UKF.

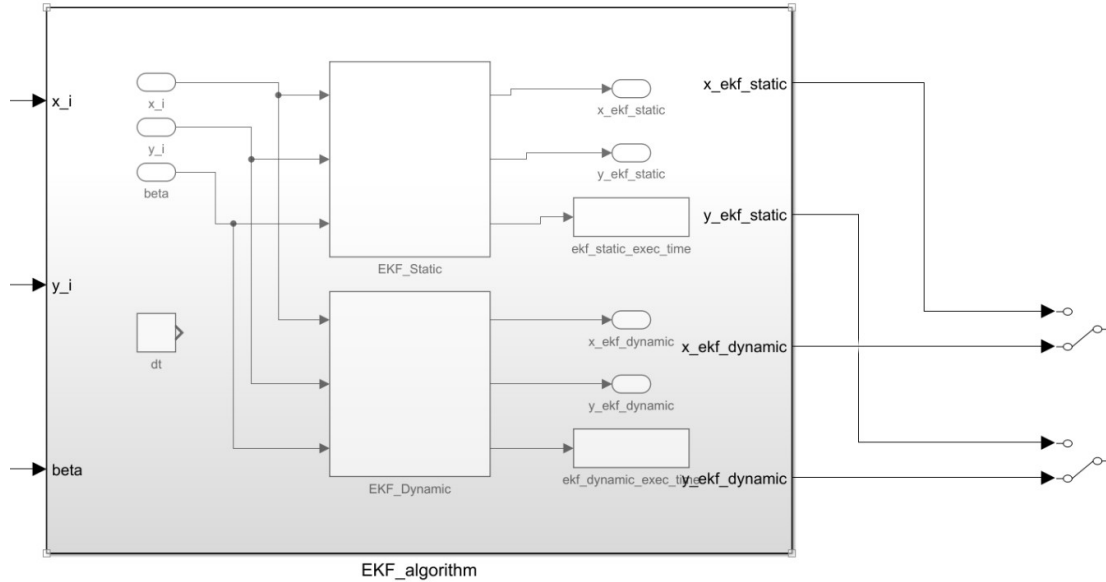


Figure 4: Simulink EKF Algorithm architecture

As illustrated in the provided diagram, the noisy observer coordinates  $(x_i, y_i)$  and the unwrapped bearing angle  $(\beta)$  are distributed simultaneously to three primary algorithm subsystems:  $RLS\_algorithm$ ,  $EKF\_algorithm$ , and  $UKF\_algorithm$ .

To ensure the system can adapt to different tracking scenarios without requiring structural changes to the model, each filter subsystem is internally divided into two parallel processing blocks. One block is strictly configured with the state matrices for a static target, while the other is configured for a dynamic target (incorporating velocity estimation).

Manual switches placed at the output of these subsystems allow the operator to seamlessly toggle between the static and dynamic estimation results depending on the active scenario. Furthermore, each block is configured to output its execution time ( $exec\_time$ ), allowing for direct quantitative comparison of the computational complexity between the linear RLS and the non-linear Kalman variants.

(Note: In the figure we can see the EKF block. The exact same architectural logic is duplicated in a parallel blocks to generate the other algorithms).

### 3.5 Performance Evaluation and Error Metrics

To accurately quantify the performance of the estimation algorithms, the Simulink model employs custom error-calculation blocks.

Our system calculates the true positional error—the absolute Euclidean distance between the estimated coordinates and the actual target coordinates. The percentage error is then calculated relative to the target's true range from the observer.

The mathematical implementation for the general positional error is defined as follows:

First, the true target range from the origin is calculated (with a mathematical safety bound of 0.001 to prevent division by zero if the target is exactly at the origin):

$$(49) \quad D_{tgt} = \sqrt{x_{tgt}^2 + y_{tgt}^2}$$

Next, the absolute positional error between the estimated state  $(x_{est}, y_{est})$  and the true target state  $(x_{tgt}, y_{tgt})$  is computed:

$$(50) \quad e_{pos} = \sqrt{(x_{est} - x_{tgt})^2 + (y_{est} - y_{tgt})^2}$$

Finally, the percentage error is extracted:

$$(51) \quad Error_{\%} = \frac{e_{pos}}{D_{tgt}} \cdot 100$$

*(Note: A parallel calculation is performed to extract the independent percentage errors for the X and Y axes).*

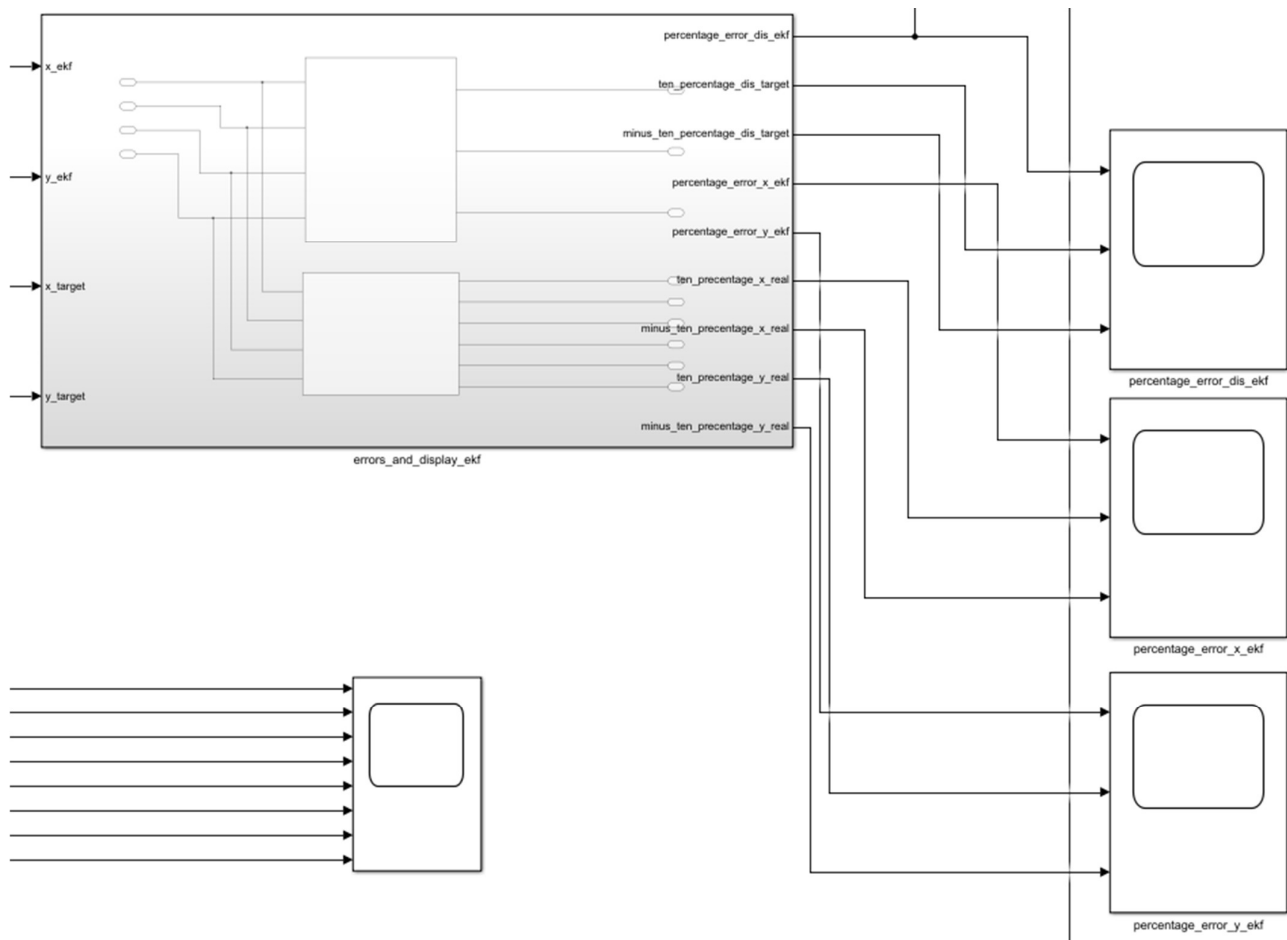


Figure 5: Simulink block diagram of the error calculation subsystem

Rather than routing all data through a single, cluttered calculation subsystem, the architecture utilizes dedicated, independent error blocks for each algorithm (RLS, EKF, UKF).

As illustrated in the provided diagram (showing the EKF subsystem as an example), the *errors\_and\_display\_ekf* block receives the true target coordinates ( $x_{target}$ ,  $y_{target}$ ) alongside the filter's estimated coordinates ( $x_{ekf}$ ,  $y_{ekf}$ ). Internally, it calculates the percentage error for the general Euclidean distance, the independent X-axis, and the independent Y-axis.

These calculations are then routed to three distinct scopes. To provide immediate visual confirmation of algorithm convergence, the function blocks also generate and output constant reference bounds at  $\pm 10\%$ , representing the project's defined success threshold. These reference lines are plotted directly alongside the active error signals on the scopes.

Furthermore, an additional central scope (visible on the lower left of the diagram) collects the raw positional data. It plots the true target coordinates

alongside the coordinate estimates from all three algorithms over time. This provides an intuitive, overarching visual comparison of the tracking trajectories before any percentage calculations are applied.

## 3.6 Complete System Architecture

When combined, the kinematic generators, the bearing calculation and unwrapping blocks, the parallel algorithm routing subsystems, and the dedicated error metric scopes form the complete *Phase 1* simulation environment. This comprehensive model allows for rapid, isolated testing of any combination of observer trajectories, target dynamics, and sensor noise profiles.

## 3.7 Simulation Results

Before analyzing the advanced kinematic scenarios, initial testing was conducted with a linear observer trajectory. These simulations perfectly confirmed the mathematical proofs established in *Chapter 2*: when the observer moves in a straight line, the bearing angle **lacks sufficient geometric variation over time**. Consequently, the system state remains unobservable, and all three estimation filters completely failed to converge.

To achieve observability, the observer must execute a non-linear trajectory. The following sections evaluate the algorithms under complex kinematics, beginning with the baseline physical testing range of tens of meters.

### 3.7.1 Scenario A: Lemniscate Observer vs. Static Target

This scenario introduces the novel figure-eight (lemniscate) observer trajectory to guarantee a continuously rich, variable bearing angle. The simulation was initialized with the following parameters:

- **Observer (Lemniscate Motion):** Initial position  $(x_0, y_0) = (40, 20)$  meters. The trajectory is governed by an angular frequency  $\omega = 0.4$  rad/s and a scale factor  $a = 20$ .
- **Target (Static):** Fixed position at  $(x_{tgt}, y_{tgt}) = (50, 60)$  meters.
- **Initial True Range:**  $\sim 41.2$  meters.

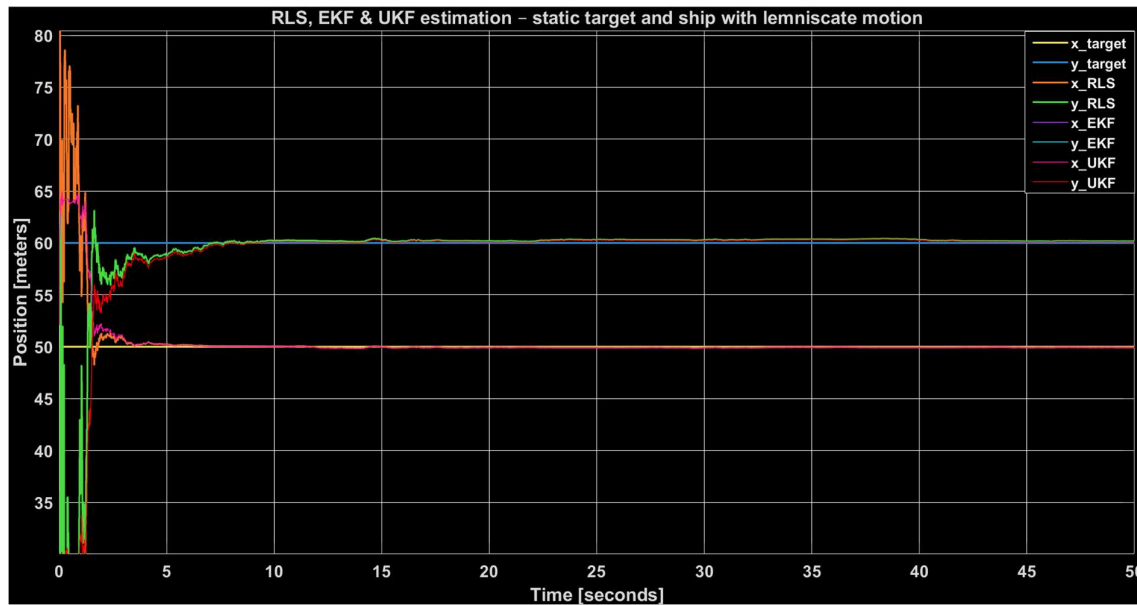


Figure 6: Position tracking comparison for RLS, EKF, and UKF in Scenario A (Static Target)

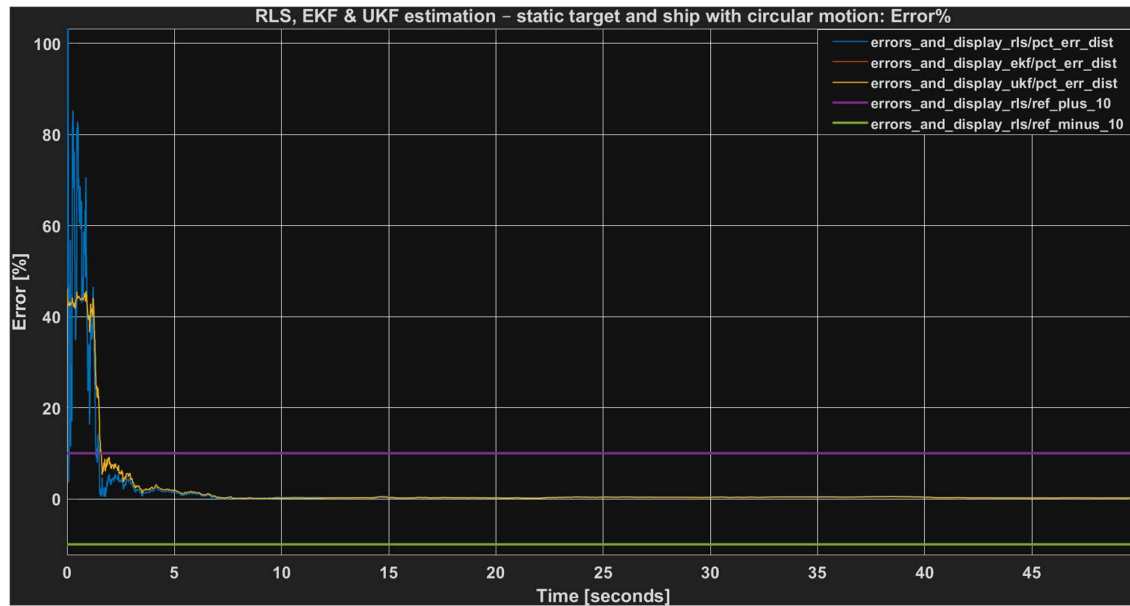


Figure 7: Distance estimation error (%) vs. Time comparison for RLS, EKF, and UKF in Scenario A (Static Target)

## Convergence and Accuracy Analysis

As shown in the position tracking graph above, all three algorithms experience a brief period of initial oscillation as the matrices populate with the first incoming noisy measurements. However, driven by the rich angular data provided by the lemniscate movement, the filters rapidly correct their state vectors and lock onto the true target coordinates of  $X = 50$  and  $Y = 60$ . It is notable that the EKF and UKF trajectories almost perfectly overlap, demonstrating nearly identical tracking efficiency for this specific static non-linear condition.

The percentage error graph quantitatively confirms the success of the system. According to the project goals, the algorithm must converge below a  $\pm 10\%$  relative error threshold within 20 seconds for a target in the tens-of-meters range. The simulation significantly outperformed this baseline. All three filters — RLS, EKF, and UKF— plunged beneath the  $\pm 10\%$  threshold within approximately 3 seconds of simulation time and maintained a stable, near-zero error state for the remainder of the test.

### 3.7.2 Scenario B: Circular Observer vs. Dynamic Target (linear movement)

To evaluate the system's performance against a moving vessel, the simulation was transitioned to the dynamic state models. Attempting to track a moving target with the static algorithms would immediately result in divergence, as the unmodeled target velocity would manifest as an overwhelming, cumulative measurement error. Consequently, this scenario utilizes the expanded 8 – state Extended and Unscented Kalman Filters, alongside the Recursive Least Squares algorithm augmented with a forgetting factor ( $\lambda$ ).

The simulation was initialized with the observer moving in a circular trajectory to maintain observability, while the target executed a constant-velocity linear maneuver:

- **Observer (Circular Motion):** Initial position  $(x_0, y_0) = (-20, 25)$  meters. The trajectory is governed by an angular frequency  $\omega = 0.6$  rad/s and radius of  $R \approx 32.02$  meters.
- **Target (Dynamic Linear Motion):** Initial position  $(x_{tgt}, y_{tgt}) = (30, 40)$  meters, moving with a constant velocity of  $v_x = 0.2$  m/s and  $v_y = 0.2$  m/s.
- **Initial True Range:**  $\sim 53.8$  meters.



Figure 8: Position tracking comparison for RLS, EKF, and UKF in Scenario B (Dynamic Target)

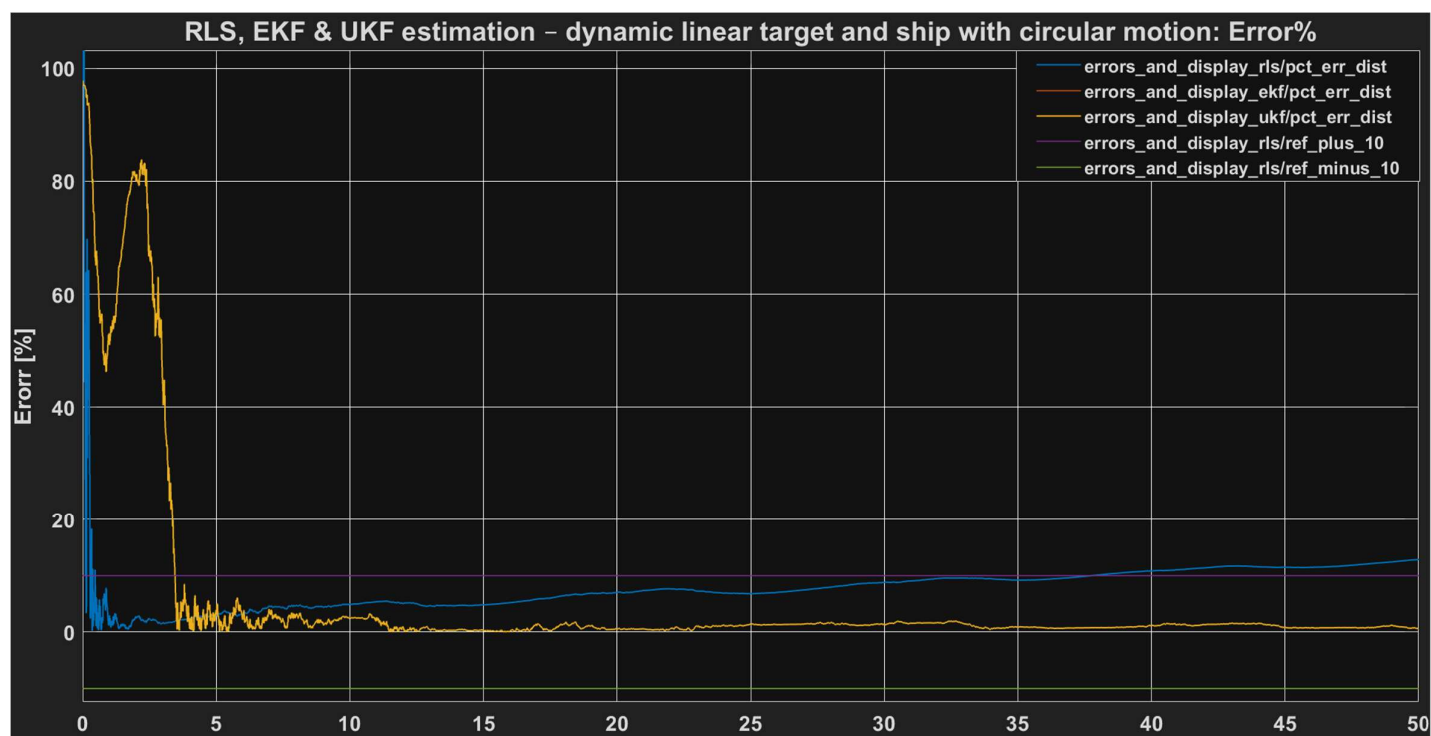


Figure 9: Distance estimation error (%) vs. Time comparison for RLS, EKF, and UKF in Scenario B (Dynamic Target)



## Convergence and Accuracy Analysis

The introduction of target dynamics reveals a stark contrast in algorithm performance. As illustrated in the position tracking scope, the true target coordinates (yellow and blue lines) increase linearly over time. After an initial transient phase where the algorithms hunt for the correct state, both the EKF and UKF successfully lock onto the moving target, with their estimated trajectories closely shadowing the true coordinates for the remainder of the simulation.

Conversely, the RLS estimator struggles significantly in the dynamic regime. The position graph shows the RLS estimates failing to smoothly track the target's advance, resulting in a continuous steady-state lag.

The fundamental architecture of the Pseudo-Linear RLS algorithm explains this failure. Unlike the Kalman variants, which utilize a discrete state-transition matrix to mathematically predict how velocity propagates into position over time, the RLS relies entirely on the exponential forgetting factor ( $\lambda$ ) to handle dynamic changes. This creates an unresolvable optimization trap: a high  $\lambda$  causes the filter to weight past geometry too heavily, creating the observed "drag" behind the moving target. Conversely, lowering  $\lambda$  makes the filter overly sensitive to the injected sensor noise, resulting in violent, erratic estimates.

This performance gap is quantified in the percentage error graph. The EKF and UKF successfully drive their estimation errors below the required  $\pm 10\%$  threshold within approximately 4 seconds—well under the 20 second goal defined for this range—and maintain that accuracy as the target moves. The dynamic RLS, however, is crippled by the  $\lambda$  trade-off; it overshoots the threshold drastically and struggles to consistently maintain an error below 10%, proving its inferiority to the robust covariance tracking of the Kalman filters in dynamic scenarios.

## Empirical Validation of Kinematic Limitations

While the EKF and UKF successfully tracked the linear dynamic target, it is mathematically critical to recognize the limitations of the underlying state model. The transition matrices formulated in *Chapter 2* operate under a strict Constant Velocity (CV) assumption.

To empirically test this boundary, supplementary simulations were executed wherein the target was programmed to perform non-linear, circular maneuvers. These tests directly validated the theoretical constraints: the unmodeled target accelerations immediately corrupted the predictive step of the algorithms. Because acceleration is not accounted for in the 8-state vectors, the predicted target positions continuously deviated from the true trajectory, causing a rapid and complete divergence of all three filters (including the EKF and UKF). This successfully demonstrates that tracking an actively maneuvering target falls outside the scope of a CV model and would

fundamentally require upgrading the system to a Constant Acceleration (CA) model or an Interactive Multiple Model (IMM) architecture.

### 3.8 Chapter Summary

In summary, the simulation phase successfully validated the theoretical boundaries of the estimation system across all operational ranges, from dozens of meters to several kilometers. The results definitively proved that system observability hinges on observer maneuverability; complex observer kinematics, such as circular or lemniscate trajectories, are strictly required to generate the angular richness necessary for range convergence. While all three algorithms performed with high, near-identical accuracy against static targets across all distances, the introduction of target dynamics exposed the limitations of the pseudo-linear approach, with the covariance-based EKF and UKF decisively outperforming the RLS. Having mapped these algorithmic behaviors, verified their microsecond-level computational efficiency, and established strict kinematic limitations, the theoretical models are fully validated and ready for deployment in the physical hardware experiments.

## 4. Implementation

### 4.1 Field Data Acquisition

To validate the theoretical target tracking algorithms (RLS, EKF, and UKF) under real-world conditions, field tests were conducted on a flat, open surface (a roller skating rink) to approximate a 2D planar environment. Data acquisition was performed using two smartphones equipped with the MATLAB Mobile application, with one device acting as the "Observer" (Ship) which was mounted on a gimbal utilizing the DJI Mimo application to actively track and follow the target, and the other as the "Target."

In target dynamic scenarios, we employed a higher velocity for the observer (e.g., via cycling or light jogging) relative to the target's walking speed. This kinematic configuration was necessary to maintain optimal geometric proximity, particularly given the target's linear trajectory. By ensuring the observer maintains a higher speed, we prevent the target from drifting out of the effective range, thereby sustaining the significant angular variation ( $\Delta\beta$ ) required to ensure system observability and robust algorithm convergence.

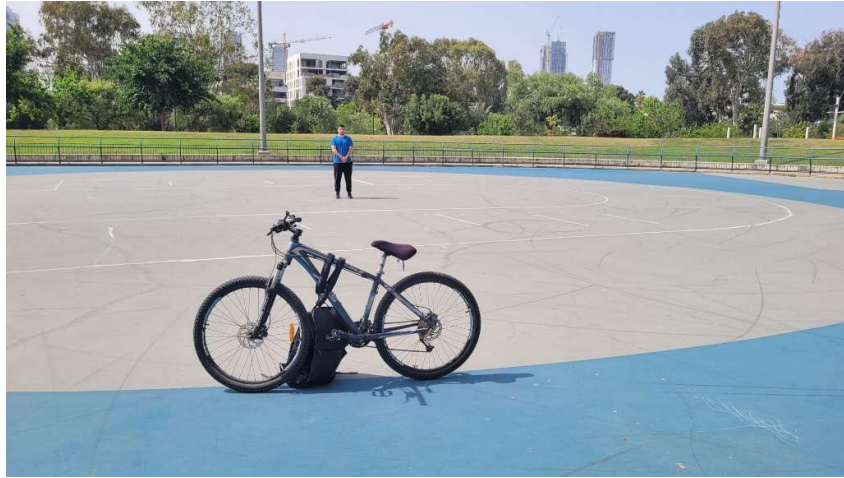


Figure 10: Field testing environment (roller skating rink) and observer platform (bicycle)

## 4.2 Hardware Limitations: Gimbal Magnetic Interference

The original implementation plan dictated that the absolute line-of-sight angle to the target would be measured directly using the smartphone's built-in magnetometer (compass azimuth). However, practical field testing revealed a critical hardware limitation.

The specific gimbal utilized for tracking employs a strong magnetic mounting connector. This magnetic proximity introduced severe, non-linear noise and a massive static bias to the magnetometer readings. When this corrupted azimuth data was fed into the system, it resulted in complete divergence of the tracking algorithms, as the recursive filters could not overcome the continuous, localized magnetic interference.



Figure 11: Smartphone observer mounted on the tracking gimbal

### 4.3 Engineering Workaround: Gyroscope Integration

Because the theoretical model was sound but the hardware was compromised, the measurement strategy was adapted to bypass the magnetometer entirely. The system was re-engineered to rely purely on inertial sensors, specifically the gyroscope, which is immune to magnetic fields.

Instead of measuring the absolute azimuth directly, the observer's relative angle to the target was calculated by integrating the angular velocity over time. The adapted software implementation, executed in the MATLAB preprocessing script, is as follows:

1. **Static Bias Removal:** At the beginning of the recording, a 5-second stationary calibration period was captured. The average angular velocity during this period was calculated and subtracted from the entire dataset to eliminate inherent hardware drift.
2. **True Initial Bearing Calculation:** The theoretical, true initial angle ( $\beta_0$ ) was calculated using the initial, smoothed GPS positions of both the observer and the target.
3. **Continuous Angle Calculation:** The corrected angular velocity ( $\omega$ ) was integrated over time using trapezoidal numerical integration (cumtrapz in MATLAB) to find the relative change in angle. This change was continuously added to the true initial angle to produce a clean, drift-resistant bearing measurement:

$$\beta_{measured}(t) = \beta_0 + \int_0^t \omega_{corrected}(\tau) d\tau$$

4. **Angle Wrapping:** The resulting angle was wrapped to the  $[-\pi, \pi]$  interval to maintain mathematical stability within the Extended and Unscented Kalman Filters.

### 4.4 Data Preprocessing and Coordinate Transformation

Because the raw data from MATLAB Mobile is sampled at varying frequencies, a synthetic 10Hz master clock was created. All sensor data, including GPS position and angular velocity, were linearly interpolated to this unified 10 Hz timeline.

To convert the observer and target locations into a usable Cartesian plane, the geodetic coordinates (Latitude, Longitude) were transformed into Earth-Centered, Earth-Fixed (ECEF) coordinates, and subsequently into a Local East-North-Up (ENU) frame in meters. A 15-sample moving average filter (representing 1.5 seconds of data) was applied to the observer's  $X$  and  $Y$

coordinates to smooth out inherent GPS inaccuracies before calculating the spatial distances.

## 4.5 Simulink Integration

Once transformed into the ENU frame, the spatial coordinates and gyroscope-derived bearings were structured into 10 Hz MATLAB timetable objects. These timetables serve as the direct inputs for the Simulink models housing the RLS, EKF, and UKF blocks. By unifying the data in this format, the simulation environment allows for a clear and consistent comparison of algorithm tracking performance.

# 5. Experimental Results and Performance Analysis

This section presents the experimental tracking results and performance analysis of the three implemented estimation filters: Recursive Least Squares (RLS), Extended Kalman Filter (EKF), and Unscented Kalman Filter (UKF). The evaluation is based on real-world measurements collected during field testing under physical noise conditions.

## 5.1 Sensor Configuration and Adaptation

During the testing phase, target azimuth measurements were required to track the target. However, physical testing revealed that the azimuth measurements suffered from severe electromagnetic interference. This was caused by the strong magnetic mount of the gimbal, which rendered the magnetometer readings highly unreliable.

To resolve this issue under physical constraints, the measurement strategy was adapted by completely bypassing the magnetometer. Instead, relative bearing angles were calculated by integrating the gyroscope's angular velocity over time. To prepare the sensor data for the tracking models, a preprocessing pipeline was executed:

- **Sensor Clock Alignment:** The raw sensor measurements were synchronized and aligned to a unified clock.
- **Coordinate Transformation:** The aligned sensor data was transformed into a local Cartesian coordinate system (East-North-Up / ENU).

Once preprocessed, the dataset was imported into the Simulink models to test and evaluate the tracking performance of the algorithms under realistic noise and gimbal dynamics.

## 5.2 Performance Comparison in Representative Scenarios

The tracking algorithms were tested using representative datasets to evaluate convergence rates, steady-state accuracy, and response to target kinematics. The measurements presented below serve as examples of the collected datasets.

### 5.2.1 Static Target Scenario (Circular Ship)

In this static target scenario, the target remains stationary while the observer moves in a circular motion around it. The tracking algorithms estimate the static position:

- Recursive Least Squares (RLS): Converged rapidly, reaching the 10% estimation error threshold in 8.02 seconds.
- Extended Kalman Filter (EKF) and Unscented Kalman Filter (UKF): Both filters converged to the true target position in 8.96 seconds.

Note on Overlap: The EKF and UKF trajectories and estimation error curves are nearly identical in this scenario. In visual plots, the EKF line is completely obscured beneath the UKF line due to their overlapping performance.

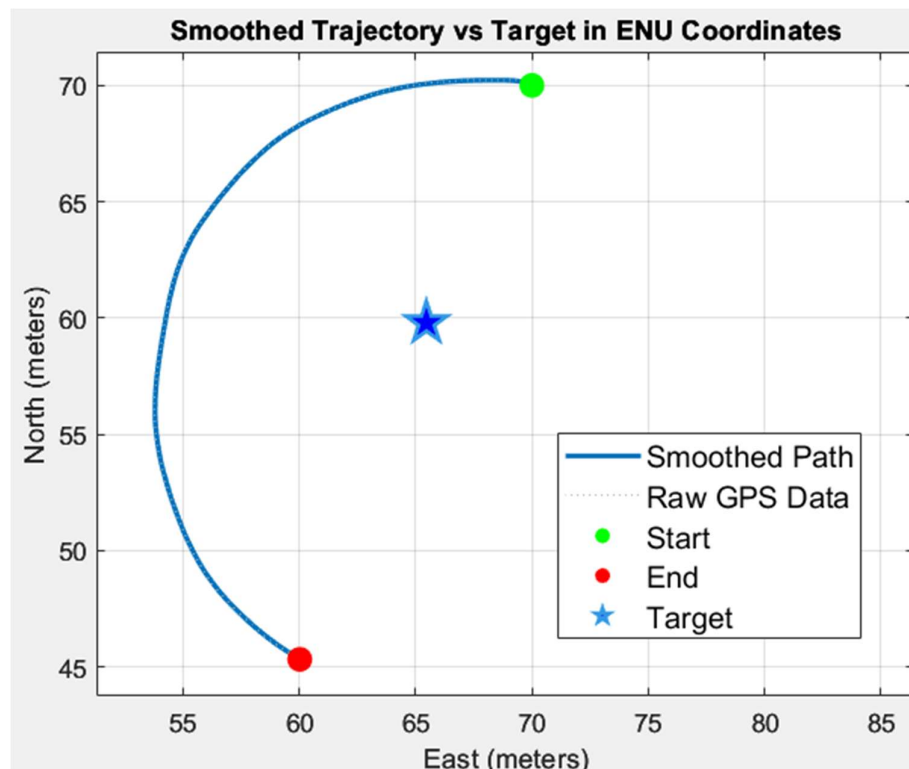


Figure 12: Smoothed observer path and true target coordinates in ENU system (Static Scenario)

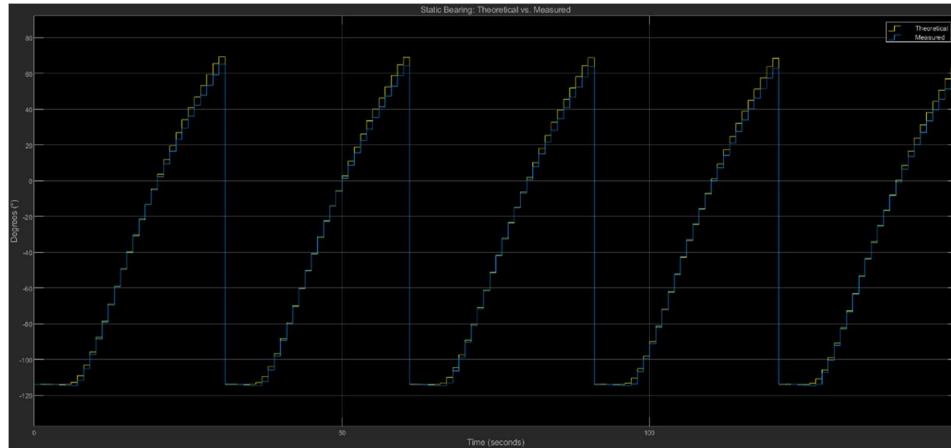


Figure 13: Measured vs. theoretical bearing angle (degrees) over time (Static Scenario)

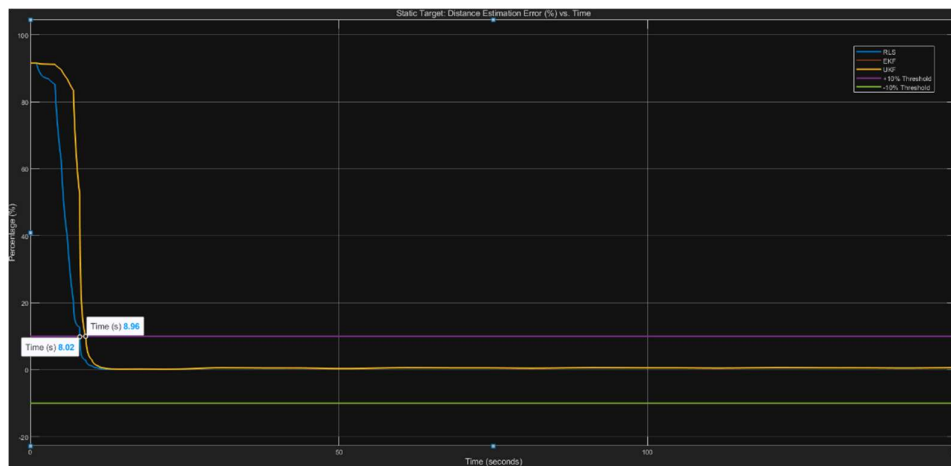


Figure 14: Distance estimation error (%) vs. Time comparison for RLS and UKF/EKF (Static Scenario)

### 5.2.2 Dynamic Target Scenario (Lemniscate Observer, Linear Target)

In this dynamic target scenario, the observer moves in a Lemniscate trajectory while the target moves in a linear trajectory:

- Recursive Least Squares (RLS): Although the RLS filter initially converged, it failed to sustain tracking over time. Accumulating estimation errors caused the filter to diverge, exceeding the 10% threshold after 30 seconds. This highlights the limitations of linear approximations for moving targets in physical environments.
- Kalman Filters (EKF & UKF): Both non-linear filters demonstrated robust tracking performance, rapidly converging below the 10% estimation error threshold in 2.06 seconds and successfully adapting to the changing kinematics of the moving target. Their trajectories overlap, confirming that both filters handle the coordinate transformations and non-linear dynamics with equivalent accuracy, maintaining an average tracking error of around 3% after convergence.

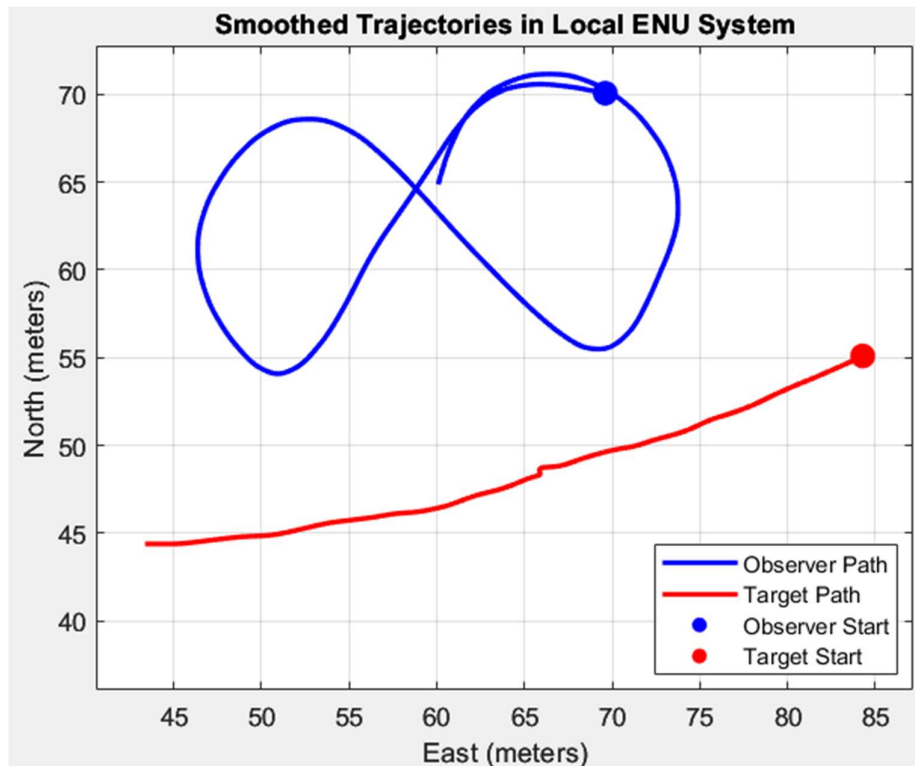


Figure 15: Figure-eight observer path and linear target path in local ENU system (Dynamic Scenario)

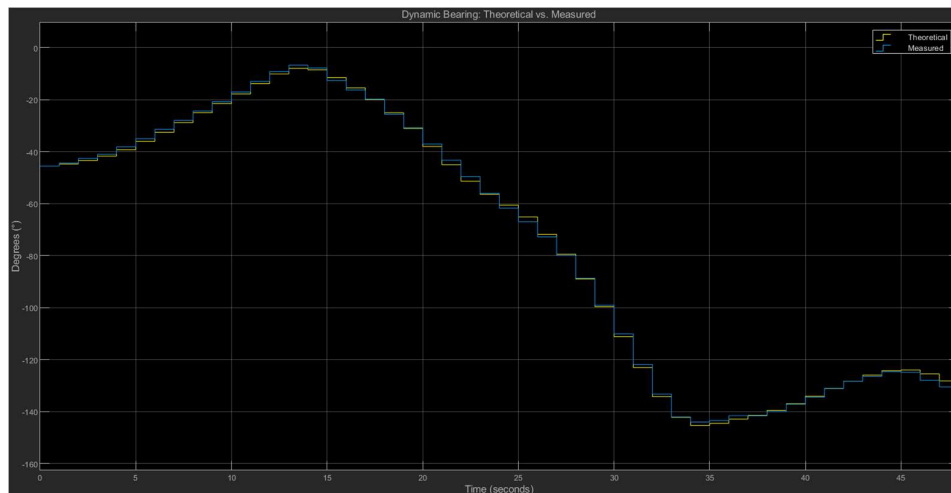


Figure 16: Measured vs. theoretical bearing angle (degrees) over time (Dynamic Scenario)



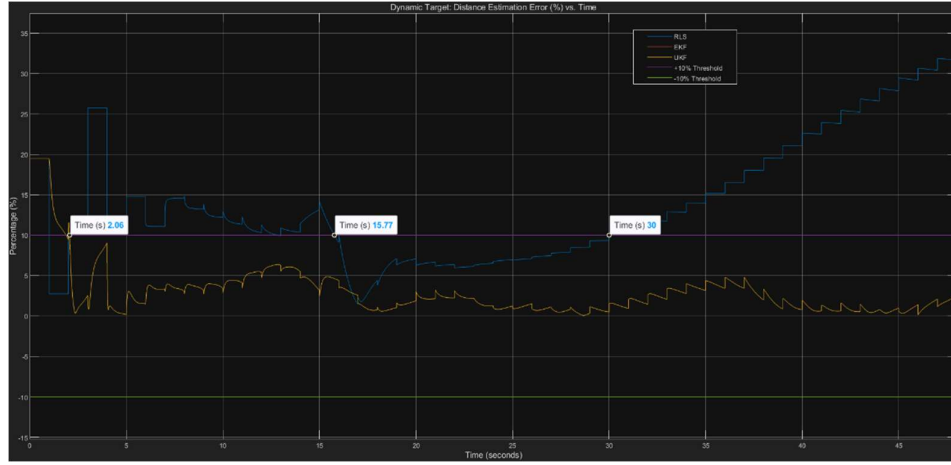


Figure 17: Distance estimation error (%) vs. Time comparison for RLS and UKF/EKF (Dynamic Scenario)

To provide a clear comparative analysis, the convergence times and CPU execution times of the three algorithms (RLS, EKF, and UKF) across all simulated and physical scenarios are summarized in Table 1.

| Target Type | Ship Trajectory | Data Source | RLS Conv. (s)               | EKF Conv. (s) | UKF Conv. (s) | CPU Time                                                 |
|-------------|-----------------|-------------|-----------------------------|---------------|---------------|----------------------------------------------------------|
| Static      | Circular        | Simulation  | 2.0s                        | 2.0s          | 2.0s          | RLS: 3.8 $\mu$ s<br>EKF: 6.4 $\mu$ s<br>UKF: 9.2 $\mu$ s |
| Static      | Circular        | Real-World  | 8.02s                       | 8.96s         | 8.96s         | RLS: 4 $\mu$ s<br>EKF: 6.1 $\mu$ s<br>UKF: 8.9 $\mu$ s   |
| Static      | Lemniscate      | Simulation  | 6.0s                        | 8.0s          | 8.0s          | RLS: 4.2 $\mu$ s<br>EKF: 7 $\mu$ s<br>UKF: 9.5 $\mu$ s   |
| Static      | Spiral          | Real-World  | 4.7s                        | 6.1s          | 6.1s          | RLS: 3.2 $\mu$ s<br>EKF: 6.4 $\mu$ s<br>UKF: 9 $\mu$ s   |
| Linear      | Lemniscate      | Simulation  | 1.0s<br>(Diverges at 38s)   | 4.0s          | 4.0s          | RLS: 3.3 $\mu$ s<br>EKF: 6.7 $\mu$ s<br>UKF: 8.9 $\mu$ s |
| Linear      | Lemniscate      | Real-World  | 15.77s<br>(Diverged at 30s) | 2.06s         | 2.06s         | RLS: 4.2 $\mu$ s<br>EKF: 6.1 $\mu$ s<br>UKF: 8.8 $\mu$ s |

Table 1: Comparison of filter convergence and CPU execution times between simulation and real-world scenarios

## 5.3 Discussion and Proof of Concept

The implementation phase underscored the crucial importance of physical testing and validation. Relying purely on theoretical sensor specifications would have overlooked the gimbal's magnetic interference, which was only identified and bypassed during real-world testing.

Ultimately, this project establishes a practical baseline and serves as a successful proof of concept for passive, bearing-only tracking systems in marine or vehicle environments.

The comparative analysis yields the following engineering trade-offs:

- **Computational Complexity:** RLS is the most computationally light algorithm, followed by EKF. UKF is the most expensive due to the propagation of sigma points, though it avoids the calculation of Jacobians.
- **Kinematic Capabilities:** Simple linear filters like RLS are suitable only for static scenarios, whereas non-linear estimators (EKF and UKF) are essential for tracking the non-linear kinematics of dynamic targets under physical noise conditions.

## 6. Summary, Conclusions, and Suggestions for Future Work

### 6.1 Summary

The main objective of this project was to implement and evaluate a passive target range estimation system that operates solely on self-position and bearing-angle measurements, thereby addressing critical naval and military security scenarios where active radar or sonar broadcasting would compromise the observer's position. The work was divided into two consecutive phases, starting with a comprehensive simulation phase in MATLAB and Simulink to evaluate the theoretical performance of the Recursive Least Squares, Extended Kalman Filter, and Unscented Kalman Filter algorithms, and then transitioning to a physical implementation phase to test these filters under real-world conditions.

To validate the system in a real-world setting, field tests were conducted in an open skating rink using two smartphones running MATLAB Mobile, where the observer phone was mounted on the gimbal and controlled via the DJI Mimo application to actively track the target, and the second phone acted as the target. Sustaining sufficient angular variation through their relative motion was crucial to ensure system observability.

During the testing phase, the azimuth was required to track the target, but the azimuth measurements suffered from severe magnetic interference due to the gimbal's strong magnetic mount. This forced an adaptation of the measurement strategy: the magnetometer was bypassed completely, and the gyroscope's angular velocity was integrated over time to calculate the relative bearing angles instead. Once the processed sensor data was aligned to a unified clock and transformed into a local Cartesian coordinate system, it was imported into the Simulink models to evaluate the tracking performance under realistic noise conditions. As examples from the collected datasets, in a static target scenario (circular observer), RLS reached the 10% estimation error

threshold in 8.02 seconds, while the EKF and UKF converged in 8.96 seconds. In a dynamic target scenario (lemniscate observer, linear target), the EKF and UKF both demonstrated robust tracking, converging below the 10% threshold in 2.06 seconds and maintaining a low tracking error (averaging 2.1% after convergence). In contrast, the RLS filter was unable to track the moving target over time. Although it initially converged, it eventually diverged, exceeding the 10% threshold after 15.77 seconds.

Furthermore, the trials confirmed critical geometric and modeling limitations. For scenarios where a linear observer path is used against a static target, or where both the observer and target move in linear trajectories, the algorithms fail to converge at all due to insufficient angular variation (lack of observability). Similarly, when the target executes non-linear (accelerating or curved) maneuvers, all three filters diverge rapidly because the underlying constant-velocity state models cannot adapt to the unmodeled target acceleration.

## 6.2 Conclusions

Although all three algorithms successfully estimated the target position under ideal simulation conditions, their performance differed significantly when subjected to real-world sensor noise and time-varying calibration errors, with each filter demonstrating distinct trade-offs in terms of convergence speed and sensitivity to initial state estimates. Both the Extended Kalman Filter (EKF) and the Unscented Kalman Filter (UKF) performed exceptionally well and yielded nearly identical tracking trajectories, with their error curves overlapping throughout both static and dynamic test scenarios. This makes both non-linear filters highly suitable for real-world bearing-only tracking, with the EKF offering a lower computational cost since it avoids the sigma-point propagation required by the UKF. In contrast, the linear Recursive Least Squares (RLS) filter, while computationally the lightest, proved highly effective for static targets but failed to track moving targets due to its inability to handle the non-linear kinematics of dynamic scenarios.

The implementation phase highlighted the critical importance of experimental validation, since theoretical assumptions about sensor reliability are often disrupted by unexpected physical conditions like the magnetic interference from the gimbal mount. Ultimately, this project serves as a successful proof of concept that establishes a practical baseline for passive tracking systems, and shows that while simpler linear approximations like Recursive Least Squares are computationally light, non-linear filters like the Extended and Unscented Kalman Filters are essential for handling the complex kinematics of moving targets in real environments.

## 6.3 Suggestions for Future Work

Future research and development could expand on this tracking system in four key areas:

- **Real-Time Embedded Implementation:** Transitioning from offline MATLAB processing to live computation on embedded hardware (e.g.,

FPGA or microcontrollers) to provide instantaneous range estimations for autonomous naval navigation and threat avoidance.

- **Computer Vision and AI Integration:** Supplementing the bearing-only system by calculating relative angles directly from video feeds. Fusing this with gyroscope data would reduce sensor errors, while deep learning models could automate target classification and maintain tracking during visual occlusions.
- **Hybrid Tracking Algorithms:** Developing an algorithm that dynamically toggles between static and dynamic state models based on target kinematics. This would prevent filter divergence and improve robustness in unpredictable environments.
- **3D Framework Extension:** Upgrading the current 2D geometric and mathematical models to three dimensions, allowing the system to track aerial targets and expanding its use in air defense and surveillance.

## 7. Documentation

The complete source code, Simulink models, and raw sensor datasets developed for this project are publicly hosted on GitHub at [https://github.com/Karakan0112/Target\\_Range\\_Estimation\\_Using\\_Self\\_Position\\_and\\_Camera\\_Angle](https://github.com/Karakan0112/Target_Range_Estimation_Using_Self_Position_and_Camera_Angle). The repository structure and its primary components are detailed below:

- Raw Sensor Logs (.mat files): Physical coordinates and gyroscope data logged via MATLAB Mobile:
  - sensorlog\_try31\_ship.mat: Static target with a circular observer trajectory.
  - sensorlog\_try62\_ship.mat: Static target with a spiral observer trajectory.
  - sensorlog\_try43\_ship.mat and sensorlog\_try43\_target.mat: Dynamic scenario featuring a linear target and a lemniscate (figure-eight) observer trajectory.
- Preprocessing MATLAB Scripts (.m files):
  - Static\_try31.m: Converts and processes raw sensor data for the static target / circular observer scenario (try31) into local metric coordinates.
  - Static\_try62.m: Converts and processes raw sensor data for the static target / spiral observer scenario (try62) into local metric coordinates.
  - Dynamic.m: Generates simulated parameters or sets up the workspace for the linear target / lemniscate observer scenario (try43).
- Simulink Estimation Models (.mdl files):

- First\_Model\_2026\_SIMULATION\_ONLY\_FINAL.mdl handles pure simulation runs under ideal conditions.
- Second\_Model\_2026\_Static\_Target\_Circ\_Ship\_WET.mdl processes real-world sensor data for static target tracking.
- Third\_Model\_2026\_Dynamic\_Target\_Circ\_Ship\_WET.mdl processes real-world sensor data for dynamic target tracking.

## 8. Bibliography

1. Aidala, V. J. (1979). Kalman filter behavior in bearings-only tracking applications. IEEE Transactions on Aerospace and Electronic Systems, AES-15(1), 29–39.
2. DJI, Osmo Mobile 6, Spes, <https://www.dji.com/uk/osmo-mobile-6/specs>
3. Islam, S. A. U., & Bernstein, D. S. (2019). Recursive least squares for real-time implementation. IEEE Control Systems Magazine, 39(3), 82–89.
4. Jiao, J., & Saldarriaga Fuertes, A. (2020). Lecture 15: Recursive least squares algorithm. EECS 225A – Statistical Signal Processing, University of California, Berkeley.
5. Kitani, K. Extended Kalman filter [Lecture slides]. Computer Vision, Carnegie Mellon University.
6. MathWorks, Comparison of 3-D coordinate systems. <https://www.mathworks.com/help/map/choose-a-3-d-coordinate-system.html>
- 36
7. MathWorks, ecef2enu: Transform geocentric Earth-centered Earth-fixed coordinates to local east-north-up. <https://www.mathworks.com/help/map/ref/ecef2enu.html>
8. MathWorks, geodetic2ecef: Transform geodetic coordinates to geocentric Earth-centered Earth-fixed. <https://www.mathworks.com/help/map/ref/geodetic2ecef.html>
9. Rhudy, M., & Gu, Y. (2013). Understanding nonlinear Kalman filters, part II: An implementation guide. Interactive Robotics Lab, West Virginia University.
10. Rowell, D. (2008). Introduction to recursive-least-squares (RLS) Adaptive Filters. MIT OpenCourseWare.
11. Greg Welch and Gary Bishop (2002). An Introduction to the Kalman Filter. Department of Computer Science University of North Carolina at Chapel Hill.